

AI 时代的研发组织提效

从 VibeCoding 到可信交付闭环

阿里云西南企业架构师团队负责人
侯宇



生产级 AI Coding 的三大信号

能力跃迁已经发生，组织治理成为新的分水岭



AI Coding 正在从 “个人开发辅助” 进入 “企业研发体系重构” 阶段。

01

模型能力进入生产级



- SWE-bench Verified 从 2024 年初约 3% 提升到 2026 年中 **80%+**
- 真实工程任务开始具备端到端处理能力

02

协议和工具生态标准化



- MCP
- Skills
- Rules
- Hooks
- 审计

Agent 可以**连接真实工程环境**，不只停留在聊天和补全

03

企业关注点转向可治理



- 项目知识
- 需求规格
- 执行隔离
- 测试验证
- Review
- 安全发布
- 价值度量

成为新的
分水岭



能写代码

个人开发辅助



能进工程体系

融入研发流程与工具链



能被组织治理

可治理 • 可复用 • 可度量

矛盾一：生码率越来越高，E2E 人月没有同步缩短

内部实测结果（截止 2026.03，自有基线）



前端

- 人均有效代码 **2.8x**
- 千行缺陷率 **↓30%**



后端

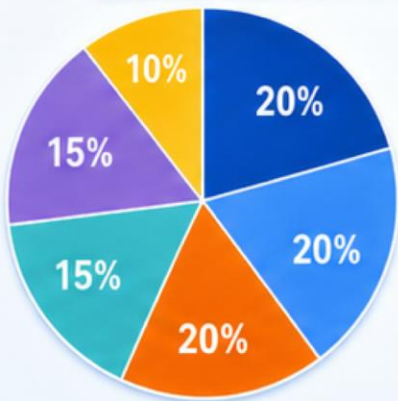
- 人均有效代码 **2.1x**
- 千行缺陷率 **↓55%**



E2E 整体

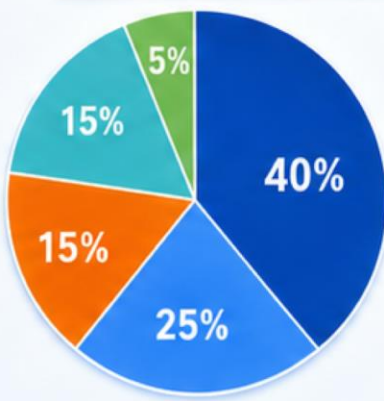
- 整体提效仅约 **30%**

第一层拆解：Coding 只占 E2E 的 20%



需求设计 / 评审 / 修改	20%
前后端联调 / 集成	20%
编码开发	20%
测试 / bugfix	15%
技术方案设计 / 评审	15%
交互视觉设计 / 评审 / 修改	10%

第二层拆解：AI 最擅长的代码约占 Coding 的 35%



核心业务逻辑代码	40%	需要业务和架构判断
系统非功能性关键代码	25%	风险高，强依赖工程约束
增删改查型业务代码	15%	AI 效果较好
前端 UI 代码	15%	AI 效果较好
测试代码	5%	AI 效果较好

$$15\% + 15\% + 5\% = 35\%$$

$$20\% \times 35\% = 7\%$$



AI 最容易发挥作用的区域，只覆盖 E2E 工作量的约 **7%**。



启发：AI Coding 可以显著提升局部效率和质量，但企业级提效必须继续穿透到 E2E 人月、环节耗时和代码类型结构。

矛盾二：Vibe Coding 很快，但存量主力系统无法直接落地

快速搭建新应用的能力，不等于企业存量生产系统的规模化交付能力



核心判断： Vibe Coding 的价值在于快速可视化想法，但不能直接等同于企业级交付能力。

Vibe Coding 适合



全新应用

从 0 到 1，快速启动



个人 Demo

验证想法，快速迭代



低稳定性要求场景

试验性、非关键场景



需求澄清和原型验证

快速可视化，缩短沟通成本

原型速度



生产能力



存量系统的硬约束



业务规则复杂

不能只看当前页面



历史债务多

代码风格和架构边界不统一



生产责任重

稳定性、性能、安全、审计、回滚都要成立



上下游一致性强

接口、状态、权限、数据都不能局部猜测



Demo 可运行不代表生产可用；新应用效率高不代表存量系统效率高。

企业级 AI 提效必须回到 Spec、API First、测试、Review、质量护栏和上下文知识工程。



开发模型选择：按复杂度选择 Agent 协同模式

简单任务可以 Vibe Coding，中等复杂度需要 Plan 先行，复杂业务必须 Spec 驱动

☆ AI Coding 不是一种固定工作方式，而是一组需要按复杂度选择的协同模式。



选择标准

局部文件，
失败后易回退？

是 → Vibe Coding

多文件、接口或测试，
规则清楚？

是 → Plan 先行

业务规则、权限、状态、
数据一致性、审计、发布风险？

是 → 可信交付



任务复杂度、协同模式和可信证据必须匹配。



简单任务走重流程：
流程成本高于变更价值



复杂 Feature 一句话 Vibe：
风险后移到联调、测试、Review、发布

解决思路：从代码生成转向可信交付链路

软件工程的思想仍然是一段时期以内的主要解决思路



研发组织提效公式：AI 能力 × 组织适配度 × 治理成熟度

模型+工具只能获得局部提效，补齐工程适配和治理成熟度才能形成端到端提效

 **核心判断** | AI Coding 的核心问题已经变化：不是有没有 AI 能力，而是组织能不能接住 AI 能力。

研发组织提效公式 **组织级研发效能** = **AI能力** × **工程适配度** × **治理成熟度**

 **这是乘法关系** 任何一个乘数接近 0，整体效果都会被拉低。



1 AI能力

- ✓ 模型
- ✓ Agent
- ✓ 工具调用
- ✓ Repo Wiki
- ✓ Quest
- ✓ Worktree



2 工程适配度

- ✓ Spec
- ✓ AGENTS.md
- ✓ Rules
- ✓ 业务知识
- ✓ 角色分工
- ✓ 存量系统上下文



3 治理成熟度

- ✓ 测试
- ✓ Review
- ✓ 安全
- ✓ 发布
- ✓ 审计
- ✓ 度量
- ✓ Credits 治理

五道门禁：把 AI 能力转化为组织适配度，再转化为治理成熟度



G1 需求门禁
能不能开工

关键资产：Spec、验收标准、反例



G2 上下文门禁
Agent 是否读懂系统

关键资产：Repo Wiki、AGENTS.md、Rules



G3 架构门禁
方案是否守住边界

关键资产：ADR、详细技术方案、任务树



G4 合入门禁
变更是否能进主线

关键资产：测试、Review、安全、Worktree



G5 发布与价值门禁
是否可上线、可复盘

关键资产：发布清单、Feature Flag、度量模型



五道门禁把 AI能力转化为工程适配度，再转化为治理成熟度；三者同时成立，个人效率才会变成组织生产力。

贯穿案例：订单详情页发起售后退款



G1 需求门禁：模糊需求不能直接进入开发和测试

问题不是“按钮怎么做”，而是“这件事现在是否具备开工条件”

从一句真实需求开始：

“在订单详情页增加申请退款按钮。”

✗ 它不是合格的开发输入，因为只表达了“要什么”，没有表达“做到什么程度才算对”。

传统需求评审的真实问题：



成本高
沟通反复
返工率高



依赖资深经验
判断标准不一
新人难以把控



经常被跳过
赶进度先开发
问题后置



结果是：需求看起来可以开工，实际只是把不确定性后移到编码、测试和发布阶段。



开工判断 | 一句话需求缺少什么



目标是否清楚

为什么要加这个入口，解决哪个业务问题



边界是否清楚

做退款申请，还是包含审核、退款、通知和财务处理



规则是否清楚

哪些场景允许，哪些场景必须拒绝



验收是否清楚

前端、接口、状态、异常路径分别如何验证



风险是否清楚

是否影响发布、回滚、审计和客服处理



需求门禁的第一步，是先判断输入是否合格。



输入评估



信息补全



判断合格



进入开发

G1 需求门禁：让 Agent 先补全需求

从一句业务诉求，到可审查、可验收、可开工的需求契约

AI 原生需求门禁的价值，是让 Agent 先把业务诉求补成可执行输入。



需求门禁把不确定性前置化，减少返工，提升交付确定性与质量。

更少返工

更快决策

更高质量

G1 需求门禁：补全后的需求契约

Agent 负责生成执行契约，人负责确认业务边界

阿里云智能集团
ALIBABA CLOUD INTELLIGENCE GROUP



合格的 Spec 不是把需求写长，而是成为设计、编码、测试、Review 和发布都能复用的统一契约。

1 业务目标

客户可在订单详情页发起退款申请

- 提升退款体验
- 减少客服咨询
- 提升问题闭环效率



2 范围边界

只支持退款申请，
不包含以下能力



换货



补发



人工赔付

3 用户路径



订单详情 申请退款 选择原因 提交 查看进度

多端一致：App / H5 / 小程序

4 后端规则

- 订单归属：仅订单本人或授权人可申请
- 订单状态：待收货 / 已完成 / 部分退款等
- 售后期限：在可申请售后时间范围内
- 幂等要求：同一订单 + 原因 + 金额幂等
- Feature Flag：按业务线 / 版本控制开关

5 状态规则

售后申请状态流转和审核边界



6 验收标准

每条业务规则都有可验证条件

- 功能：入口展示、申请流程、进度查询正常
- 规则：订单状态、期限、金额校验正确
- 接口：幂等有效、错误码完整、超时可控
- 异常：超期、越权、重复提交有明确提示
- 兼容：多端一致，Feature Flag 生效可控
- 性能：接口 P95 < 500ms，错误率 < 0.1%

7 反例与发布风险

反例（必须拦截）

- 越权申请：非本人或未授权
- 超期申请：超出售后期限
- 重复提交：同原因重复申请
- Feature Flag 关闭：不展示入口

发布风险与控制

- 灰度范围：按地域 / 客群 / App 版本
- 观察指标：申请成功率 / 失败率 / 退款时长
- 回滚方案：关闭开关 / 降级逻辑 / 数据回补
- 审计留痕：操作记录 / 状态变更 / 通知日志

⚠ 不确定即不发布，风险可控再放量



此契约作为：需求评审、方案设计、编码实现、测试用例、Code Review、发布评审的一致依据

G1 需求门禁: Live Demo 让需求、设计、交互左移

需求契约明确后，业务方可以先生成可交互 Demo，用真实体验验证需求是否成立

 需求确定后，不必等研发排期才第一次看到产品形态。

 基于 Spec、验收标准和反例，Agent 先生成完整可交互 Live Demo。

 提前验证用户路径、页面信息、异常提示和业务反馈。



Live Demo 左移三件事

 **需求确认**
从文字评审变成点击验证

 **交互验证**
开工前确认入口、表单、状态反馈、错误提示

 **设计对齐**
减少业务、产品、设计、研发之间的转译



Live Demo 可以作为需求门禁的最后一个环节。

不能替代 Spec Review、技术方案和生产代码；提前暴露体验问题、字段缺口和业务反馈。

G2 上下文门禁：Agent 必须先理解存量系统



Agent 最容易犯的错:

只读当前 Controller/Service/页面 → 模仿已有代码快速生成 → 没读到业务术语、状态流转、权限规则和历史架构决策



Repo Wiki 的价值:

不是“生成文档”，而是形成 Agent 可持续消费的工程知识底座

- 仓库结构
- 业务模块
- 关键调用链
- 历史约束



存量系统提效的第一步，不是让 Agent 写代码，而是让 Agent **读懂系统**。

进入存量系统前需要理解的上下文

上下文		对售后退款的意义
	业务术语	订单、售后、退款、审核语义必须一致
	状态流转	订单状态和售后状态不能随意跳转
	权限边界	客户、客服、运营、管理员范围不同
	接口约束	错误码、幂等、返回结构等既有约束
	发布规则与历史约束	Feature Flag、灰度、回滚、历史踩坑

G2 上下文门禁：AGENTS.md / Rules 把团队经验写进系统



四类资产边界（最佳实践）

资产	管什么	不管什么
Spec	一次需求的目标、范围、验收、反例	不写实现细节
详细技术方案	一次实现的 API、状态机、权限、发布	不沉淀长期规则
Rules	所有同类需求必须遵守的长期约束	不替代本次 Spec
AGENTS.md	工作说明、必读材料、验证命令	不承载一次性需求

售后退款沉淀出的 Rules 示例：

后端必须重新校验权限/状态/期限/幂等；状态不能绕过服务层直接修改；涉及权限、发布、CI 的变更必须跑安全扫描。

价值：把团队工作方式变成 Agent 默认行为，而不是每次靠人重复提醒。

冷启动三步

1



Repo Wiki 自动生成

- 工具扫描仓库，生成结构化文档并随代码变更持续维护

2



AGENTS.md / Rules 增量沉淀

- 不求一次写全，从真实任务中把约束、禁用写法、验证命令逐条写入，越用越准

3



业务术语与调用链校准

- 熟悉系统的工程师校准术语、状态机、权限边界、历史禁忌——老员工经验变成可读取资产

构建成本量级

仓库规模		冷启动投入	
	中小仓库 (十万行内)		自动生成为主，校准以天计
	多模块系统 (十万至百万行)		按模块分批，优先试点链路，以周计
	超大仓库 (百万行以上)		不做全量，按业务域逐域推进 + `.qoderignore`



可选增强：阿里云 RDS ContextDB

- 跨 Agent 共享的长期记忆与知识库，非冷启动前提条件



冷启动投入的每一分，都会在后续每一次任务中**复利**。

售后退款详细技术方案要明确的八件事

模块	要明确什么
 API 设计	endpoint、请求参数、返回结构、错误码、幂等 key
 领域模型	新增售后申请对象，避免把退款状态塞进订单主模型
 状态机	SUBMITTED → REVIEWING → APPROVED / REJECTED
 权限设计	前端入口可见性、后端订单归属校验、客服审核权限
 幂等设计	重复点击、网络重试、并发请求只创建一条有效申请
 审计事件	创建、审核、拒绝、失败、关闭都记录操作者和原因
 发布方案	Feature Flag、灰度范围、观察指标、回滚动作
 测试映射	每个验收标准对应单测、接口测试、E2E 和反例



技术方案确认后，任务树才有**稳定边界**；否则拆任务只是把**不确定性分散**到多个分支。



G3 架构门禁：ADR 让 Agent 不再重复踩架构坑



危险的不是明显错误，而是“合理捷径”。

为了最快跑通“退款进度展示”，Agent 倾向于：



但在存量系统中的长期代价：

风险	说明
模型耦合	售后流程塞进订单主模型，越来越臃肿
状态混乱	后续换货、补发、拒绝、申诉难扩展
审计不足	只落字段，缺事件记录和责任追踪
回滚困难	数据迁移和状态修复变复杂



ADR 记录的不是“怎么写代码”，而是架构取舍：

- 为什么不能只加字段
- 为什么走独立售后申请
- 为什么需要状态流转和事件审计



架构判断必须成为 Agent 可读取的证据，否则 AI 永远选择局部最短路径。

G3 架构门禁：任务树和 Checkpoint 让 Agent 小步并行执行

任务拆解要服务于验证，而不是只服务于分工



任务	Agent 执行内容	Checkpoint / 通过证据
T1 API 层	实现退款申请接口	接口测试草案通过； 幂等 key 和错误码明确
T2 状态模型	实现售后状态流转	状态机符合 ADR； 领域服务测试通过
T3 后端逻辑	实现权限、期限、 重复申请校验	单测、接口测试、 反例测试通过
T4 前端入口	实现按钮、表单、 状态展示	Flag 关闭时入口不可见； E2E 草案覆盖
T5 测试评测	补齐验收标准和 反例验证	验收标准到测试 映射完整
T6 Review/发布	生成 Review 说明和 发布清单	影响面、回滚、 观察指标明确



拆得好的任务：Agent 容易执行、测试容易覆盖、Review 容易判断、发布容易回滚。（任务树可由 Qoder Wake 承接自动化执行，设计需团队预先完成）

G4 架构门禁：7×24 执行，高效承接自动化编排

为什么需要 7×24 执行



任务树确认后，T1-T5 的执行
不需要工程师全程盯屏



白天做高价值判断（需求、架构、验收），
执行和验证交给 Agent 跑



长任务、批量任务不再受工作时间约束，
可以排队跨夜推进

Qoder Wake：7×24 长期运行的 AI 员工运行时（产品内置）

能力	说明
 任务编排	按任务树的顺序和依赖关系自动推进 T1-T6
 长期运行	任务可跨夜持续执行，不依赖工程师会话在线
 Checkpoint 停靠	每个任务完成后停在 Checkpoint，留下可检查证据
 结果可复盘	执行结果、测试输出、失败原因次日集中复盘

售后退款的一晚



下班前
确认任务树



Wake 依次推进
T1 API



T2 状态模型



T3 后端逻辑
(T4 前端并行)



每个任务停靠
Checkpoint 并
留下证据



次日工程师
只看 Checkpoint 证据
和失败项，不重看过程

无人值守的三条边界

1



任务树与 Checkpoint 必须企业预先设计——
Wake 承接的是执行编排，不是规划

2



高危操作（删除、发布、权限变更）仍必须
经过 Hooks 拦截和人工确认

3



“黑灯”模式跑的是执行，不是审批：
夜间产生的变更同样必须通过同一套合入证据
才能回主线

G4 合入门禁：隔离工作区执行，证据合格才回主线



前提：Spec、ADR、技术方案、任务树已确认。
G4 控制的是——在哪里写、哪些可并行、凭什么回主线。

售后退款执行编排



G4 的四类合入证据



并行不等于同时合入

前端、后端、测试可以在不同 Worktree 推进，但只有通过**同一套证据门禁**的变更才能回主线。



Agent 自主性越强，越**不能直接污染主干**。

G4 合入门禁：测试证据，验收标准覆盖闭环

没有映射到验收标准的测试，很难证明业务正确



测试证据回答：是否满足 Spec 和验收标准？



验收标准

- 1 客户只能操作自己的订单
- 2 Feature Flag 关闭拒绝申请
- 3 进行中申请不能重复创建
- 4 超过售后期限不能申请
- 5 提交成功记录审计事件



API



API 权限测试

—



接口重复提交



API 反例

—



单测

—



前端可见性



领域服务单测



领域服务单测



后端单测



E2E



E2E 越权反例



后端接口拒绝

—

—

—



审计

—

—

—

—



审计日志校验



只测按钮可见、接口 200、happy path，不能证明业务规则成立。



业务正确 > 页面可用

G4 合入门禁：评测资产，业务规则评测集沉淀正例和反例

业务规则评测集（Eval Suite）不是本次测试的重复，而是让同类需求以后有稳定验收尺子



测试证据
证明
“这次改动能不能合入”



业务规则评测集
解决另一个问题

- 同类需求以后还会来
- Agent 以后还会改订单和售后
- 关键业务规则不能每次重新解释

售后退款业务规则评测集

评测样例	稳定期望
 可售后订单提交退款申请	 创建售后申请成功
 客户访问他人订单发起退款	 必须拒绝
 不允许退款的订单状态	 必须拒绝
 已有进行中售后申请	 不能重复创建
 Feature Flag 关闭	 前端入口不可见，后端接口拒绝
 提交成功后查询审计记录	 必须存在创建事件

Test / 业务规则评测集 / E2E 的边界

机制	解决的问题
 Test	本次实现是否满足验收标准
 业务规则评测集	同类需求的关键规则是否可重复评测
 E2E	真实用户路径是否跑通



业务规则评测集是**组织资产**。每次售后退款类需求结束后，都应该把新的**正例和反例**沉淀进去。

G4 合入门禁：Code Review 多模型分层，人不做 code 级兜底

生成、审查、安全三类角色必须分离，人的判断上移到分歧和治理



* CodeReview要和需求、技术方案、ADR、测试证据等贯穿打通。

逐行 Review
会成为新瓶颈；
人的判断上移。

售后退款 Review 只抓三类高风险

- 业务规则漏实现
- 状态 / 幂等破坏边界
- 权限 / 审计 / 回滚无证据

G4 合入门禁：安全工具链清单

复用企业已有工具，让安全扫描结果成为合入证据



P0 安全问题：Agent 先修复



不能默认自动放行



安全结果必须成为合入证据

G4 合入门禁：安全治理不仅是 SAST，更是 Agent 行为治理

AI Coding 的安全边界从代码漏洞扩展到工具调用、权限范围和审计链路



传统安全工具仍然重要；Agent 时代还必须治理执行过程。

G5 发布门禁：发布不是上线按钮，而是证据链

代码合入不等于交付完成，真正完成是变更稳定运行在生产环境

AI Coding 提高开发速度后：

- 发布频率上升
- 变更数量上升
- 风险更容易在上线后集中爆发

发布、灰度、监控和回滚必须同步升级。

典型发布偏差

- 功能已合入
- 没有 Feature Flag
- 没有观察指标
- 没有回滚条件
- 没有数据修复方案

代码完成，不等于发布风险被管理。

售后退款发布证据链



Agent 可以辅助生成发布清单、影响分析、回滚方案和复盘模板；
灰度执行、监控告警和回滚操作 **必须由现有发布平台承接。**



发布平台



监控告警



灰度控制



回滚执行



发布治理的价值，是把上线经验变成 Agent 可读取、流程可校验、事故可追溯的组织能力。

G5 价值门禁：用四层效能模型度量真实价值

阿里云智能集团
ALIBABA CLOUD INTELLIGENCE GROUP

不能只看 AI 生码率，也不能把 Demo 效率等同于企业级交付能力



AI过程层落地路径：现有系统起步，轻量连接，按需平台化

先证明四类核心指标能够运行并产生管理价值，再决定自动化和平台投入

 **整体思路：** 指标先行、现有系统承载、轻量关联补齐、规模化后按需平台化。



阶段	需要建设的能力	是否需要新平台	交付结果
 第一阶段： 基线验证	 Wiki / 需求模板 最小字段  CSV  只读 API  人工确认和 现有 BI 报表	 不需要	 四类指队形成现状基线， 数据缺口和暂不可用原因明确
 第二阶段： 轻量关联	 Work Item ID  PR 必填字段  Release Manifest  采集脚本  关系表和 定时计算	 不需要独立平台； 可在现有数据环境中 增加轻量服务	 指标能下钻到需求、PR、 发布、缺陷和事故证据
 第三阶段： 规模化运营	 事件账本  幂等采集  指标语义层  权限审计  数据质量和 流程分析	 优先复用现有数据仓库、 BI 或 DevOps 平台； 无法承载时才建独立平台	 同一口径可在多团队复用， 指标可追溯、可重算、 可持续运营

什么时候才需要平台化

 多个团队需要共用同一套指标，
数据源和接口数量持续增加。

 人工维护、定时脚本和关系表
已无法稳定支撑指标重算、
版本管理和历史追溯。

 存在复杂的权限、脱敏、审计
和数据留存要求，现有基础设
施无法承载。

 方案要求建设的是**数据闭环能力**，
不预设必须建设一个新平台。



用一个 Work Item ID 串起真实交付流程

在需求、PR、发布、缺陷和业务结果中写入同一个稳定 ID

ID

Work Item ID 是一项需求在鏵个交付周期中不变的唯一编号，例如 `REQ-2026-001285`。
由需求平台生成，编号本身不编码部门、人员和状态。



环节	最小动作	形成的关系
需求 / 设计	Wiki 生成 Work Item ID，需求中保存 Figma 链接	Work Item → Design
代码	PR 必填 Work Item ID，Commit footer 辅助校验	Work Item → PR → Commit
发布	Release Manifest 写入本次发布的 Work Item 清单	Work Item ↔ Release
测试 / 生产质量	TB / Jira 记录 Work Item 和 Release； 事故复盘登记 Release ID	Work Item ↔ Release ↔ Bug / Incident
业务结果	BI 或业务负责人按 Work Item ID 回写实际结果	Work Item ↔ Business Metric

三条必须执行的规则

- 1

PR 的 Work Item 字段由合并门禁校验；
结构化字段才是正式关联，分支名和文本匹配只是候选。
- 2

支持多对多：一个 Work Item 可对应多个 PR 和 Release，
一个 Release 也可包含多个 Work Item；
子任务通过父子链聚合。
- 3

源系统继续保存权威事实；
关联层只保存 ID、关键时间、证据链接和关系。

G5 价值门禁：Credits 治理是工程治理，不是财务压缩



典型反模式：

让 Agent 在不清楚需求、不理解仓库、没有质量门禁的情况下反复试错。



Credits 浪费是五道门禁的成本反馈

浪费来源	对应缺口	治理动作
 反复读仓库	G2 不足	 Repo Wiki、`.qoderignore`、Rules
 边聊边做	G1 不足	 先 Spec、验收标准、反例
 大任务跑偏	G3 不足	 技术方案、任务树、Checkpoint
 反复修 bug	G4 不足	 测试、评测集、Review
 发布后返工	G5 不足	 Feature Flag、发布清单、回滚计划

容易被忽视的省 Credits 细节

-  **发现跑偏立即 Stop：**
不要等 Agent 自己绕回来，越跑偏越贵
-  **用 Revert / Reject / Git 回滚：**
不要用聊天方式让 AI“改回去”
-  **MCP 按需启用：**
不用的工具及时关闭，避免描述长期占用上下文
-  **指令写进 AGENTS.md：**
项目级期望行为一次写清，减少重复解释
-  **代码仓库对 AI 友好：**
控制单文件长度、补齐类型、过滤生成目录和大文件

五道门禁背后的组织资产

阿里云智能集团
ALIBABA CLOUD INTELLIGENCE GROUP

一次交付结束后，留下的不只是代码，还应该留下可复用能力



资产越完善，下一次**越快、越稳、越安全**

方案边界：工具提供能力，组织定义标准

AI Coding 的落地边界，不是“工具能不能做”，而是“谁定义规则，谁承接结果”



边界不清： Repo Wiki $\neq$ 知识治理 | Hooks $\neq$ 安全体系 | Usage 数据 $\neq$ ROI 结论

组织角色变化：人从写代码上移到定义目标、约束和验收

AI 改变的不只是个人编码方式，而是产研协作中的责任分布

人的工作重心：从执行迁移到定义、编排、判断和沉淀

	过去：执行型分工		现在：责任上移
 工程师	完成功能开发，修复问题，提交代码	➡	 定义 Spec  拆任务  编排 Agent  判断结果
 Tech Lead	设计方案，指导实现，合并代码	➡	 架构边界  Rules  Checkpoint  Review 升级标准
 测试	编写用例，执行测试，提缺陷	➡	 验收标准  评测集  E2E 门禁
 安全 / 发布 / 管理者	后置审查，发布审批，过程监管	➡	 前置规则  证据链  组织资产复用

人的价值上移

 定义目标

 设计边界

 拆解任务

 判断取舍

 沉淀规则

评价方式也要变化

 不看代码行数

 不只看 PR 数

 不只看提交次数

➡  看可信交付结果

门禁分级：不是所有需求都走满流程，但关键证据不能缺

门禁不是增加流程负担，而是按风险决定证据厚度

分级原则：



低风险轻量化



中风险标准门禁



高风险完整链路

无论流程多轻，四类证据不能缺

01

验收标准

防止 Agent 猜需求



02

上下文读取

防止破坏存量系统



03

隔离执行

防止直接污染主线



04

合入证据

保留测试、Review
和安全记录



高风险链路示例



资金



权限



状态



幂等



审计



回滚



售后退款涉及资金、权限、状态、
幂等、审计和回滚——
属于应走完整门禁的**高风险链路**。

AI

AI 的作用不是让门禁消失，而是把**需求补全、方案检查、测试生成、Review 预审和证据整理自动化**，
让高质量流程不再靠人工堆时间。

把 AI Coding 从个人效率变成组织能力

阿里云智能集团
ALIBABA CLOUD INTELLIGENCE GROUP

AI Coding 已经进入生产级阶段。

真正拉开差距的，不是某一次代码生成有多快，
而是组织能否把 AI 纳入自己的研发标准、知识资产和治理流程。

► AI Coding 的组织提效，本质是把个人工具使用升级为可治理的研发生产体系。



// 最后只看三件事：

01

交付是否更可信



质量可控

安全合规

稳定可靠

02

结果是否可度量



效率提升

成本优化

价值可见

03

经验是否可复用



知识沉淀

流程固化

持续进化

THANKS